

- von Mises, R. 1964, *Mathematical Theory of Probability and Statistics* (New York: Academic Press), pp. 394ff.[3]
- Levinson, N., Appendix B of N. Wiener, 1949, *Extrapolation, Interpolation and Smoothing of Stationary Time Series* (New York: Wiley).[4]
- Robinson, E.A., and Treitel, S. 1980, *Geophysical Signal Analysis* (Englewood Cliffs, NJ: Prentice-Hall), pp. 163ff.[5]
- Bunch, J.R. 1985, "Stability of Methods for Solving Toeplitz Systems of Equations," *SIAM Journal on Scientific and Statistical Computing*, vol. 6, pp. 349–364.[6]
- de Hoog, F. 1987, "A New Algorithm for Solving Toeplitz Systems of Equations," *Linear Algebra and Its Applications*, vol. 88/89, pp. 123–138.[7]

2.9 Cholesky Decomposition

If a square matrix $\mathbf{A}$ happens to be symmetric and positive-definite, then it has a special, more efficient, triangular decomposition. *Symmetric* means that $a_{ij} = a_{ji}$ for $i, j = 0, \dots, N-1$, while *positive-definite* means that

$$\mathbf{v} \cdot \mathbf{A} \cdot \mathbf{v} > 0 \quad \text{for all vectors } \mathbf{v} \quad (2.9.1)$$

(In Chapter 11 we will see that positive-definite has the equivalent interpretation that $\mathbf{A}$ has all positive eigenvalues.) While symmetric, positive-definite matrices are rather special, they occur quite frequently in some applications, so their special factorization, called *Cholesky decomposition*, is good to know about. When you can use it, Cholesky decomposition is about a factor of two faster than alternative methods for solving linear equations.

Instead of seeking arbitrary lower and upper triangular factors $\mathbf{L}$ and $\mathbf{U}$, Cholesky decomposition constructs a lower triangular matrix $\mathbf{L}$ whose transpose $\mathbf{L}^T$ can itself serve as the upper triangular part. In other words we replace equation (2.3.1) by

$$\mathbf{L} \cdot \mathbf{L}^T = \mathbf{A} \quad (2.9.2)$$

This factorization is sometimes referred to as "taking the square root" of the matrix $\mathbf{A}$, though, because of the transpose, it is not literally that. The components of $\mathbf{L}^T$ are of course related to those of $\mathbf{L}$ by

$$L_{ij}^T = L_{ji} \quad (2.9.3)$$

Writing out equation (2.9.2) in components, one readily obtains the analogs of equations (2.3.12) – (2.3.13),

$$L_{ii} = \left(a_{ii} - \sum_{k=0}^{i-1} L_{ik}^2 \right)^{1/2} \quad (2.9.4)$$

and

$$L_{ji} = \frac{1}{L_{ii}} \left(a_{ij} - \sum_{k=0}^{i-1} L_{ik} L_{jk} \right) \quad j = i+1, i+2, \dots, N-1 \quad (2.9.5)$$

If you apply equations (2.9.4) and (2.9.5) in the order $i = 0, 1, \dots, N-1$, you will see that the L 's that occur on the right-hand side are already determined by the time they are needed. Also, only components a_{ij} with $j \geq i$ are referenced. (Since $\mathbf{A}$ is symmetric, these have complete information.) If storage is at a premium, it is possible to have the factor $\mathbf{L}$ overwrite the subdiagonal (lower triangular but not including the diagonal) part of $\mathbf{A}$, preserving the input upper triangular values of $\mathbf{A}$; one extra vector of length N is then needed to store the diagonal part of $\mathbf{L}$. The operations count is $N^3/6$ executions of the inner loop (consisting of one multiply and one subtract), with also N square roots. As already mentioned, this is about a factor 2 better than LU decomposition of $\mathbf{A}$ (where its symmetry would be ignored).

You might wonder about pivoting. The pleasant answer is that Cholesky decomposition is extremely stable numerically, without any pivoting at all. Failure of the decomposition simply indicates that the matrix $\mathbf{A}$ (or, with roundoff error, another very nearby matrix) is not positive-definite. In fact, this is an efficient way to test *whether* a symmetric matrix is positive-definite. (In this application, you may want to replace the `throw` in the code below with some less drastic signaling method.)

By now you should be familiar with, if not bored by, our conventions for objects implementing decomposition methods, so we list the object `Cholesky` as a single big mouthful. The methods `elmult` and `elsolve` perform manipulations using the matrix $\mathbf{L}$. The first multiplies $\mathbf{L} \cdot \mathbf{y} = \mathbf{c}$ for a given $\mathbf{y}$, returning $\mathbf{c}$. The second solves this same equation, given $\mathbf{c}$ and returning $\mathbf{y}$. These manipulations are useful in contexts such as multivariate Gaussians (§7.4 and §16.5) and in the analysis of covariance matrices (§15.6).

```
struct Cholesky{
```

`cholesky.h`

Object for Cholesky decomposition of a matrix $\mathbf{A}$, and related functions.

```
    Int n;
    MatDoub el;                Stores the decomposition.
    Cholesky(MatDoub_I &a) : n(a.nrows()), el(a) {
        Constructor. Given a positive-definite symmetric matrix a[0..n-1][0..n-1], construct
        and store its Cholesky decomposition,  $\mathbf{A} = \mathbf{L} \cdot \mathbf{L}^T$ .
        Int i,j,k;
        VecDoub tmp;
        Doub sum;
        if (el.ncols() != n) throw("need square matrix");
        for (i=0;i<n;i++) {
            for (j=i;j<n;j++) {
                for (sum=el[i][j],k=i-1;k>=0;k--) sum -= el[i][k]*el[j][k];
                if (i == j) {
                    if (sum <= 0.0) A, with rounding errors, is not positive-definite.
                        throw("Cholesky failed");
                    el[i][i]=sqrt(sum);
                } else el[j][i]=sum/el[i][i];
            }
        }
        for (i=0;i<n;i++) for (j=0;j<i;j++) el[j][i] = 0.;
    }
    void solve(VecDoub_I &b, VecDoub_O &x) {
        Solve the set of n linear equations  $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ , where a is a positive-definite symmetric matrix
        whose Cholesky decomposition has been stored. b[0..n-1] is input as the right-hand side
        vector. The solution vector is returned in x[0..n-1].
        Int i,k;
        Doub sum;
        if (b.size() != n || x.size() != n) throw("bad lengths in Cholesky");
        for (i=0;i<n;i++) {
            Solve  $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$ , storing  $\mathbf{y}$  in  $\mathbf{x}$ .
            for (sum=b[i],k=i-1;k>=0;k--) sum -= el[i][k]*x[k];
            x[i]=sum/el[i][i];
        }
        for (i=n-1;i>=0;i--) {
            Solve  $\mathbf{L}^T \cdot \mathbf{x} = \mathbf{y}$ .
            for (sum=x[i],k=i+1;k<n;k++) sum -= el[k][i]*x[k];
            x[i]=sum/el[i][i];
        }
    }
    void elmult(VecDoub_I &y, VecDoub_O &b) {
        Multiply  $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$ , where  $\mathbf{L}$  is the lower triangular matrix in the stored Cholesky decom-
        position. y[0..n-1] is input. The result is returned in b[0..n-1].
        Int i,j;
        if (b.size() != n || y.size() != n) throw("bad lengths");
        for (i=0;i<n;i++) {
            b[i] = 0.;
            for (j=0;j<=i;j++) b[i] += el[i][j]*y[j];
        }
    }
}
```